Let's review our docker-compose.yml file:

```
version: '3'

services:

  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - .:/usr/src/app
```

Let's modify it to include a new service that we'll call redis:

```
version: '3'

services:

  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - .:/usr/src/app

➤   redis:
➤     image: redis
```

The definition for our new redis service is quite different from that of our web service. For a start, it's a lot simpler; it only has a single property called image.

When defining a service, there are two ways to specify the image to be used for creating containers. Our web service uses the build property to instruct Compose to build our custom image from a Dockerfile. However, to use a preexisting image instead, we can specify the image's name with the image property. Here we specify the redis image, just like in our docker run command.

Other than this, the main difference is what we *don't* specify.

We don't publish any ports. Our web service needs a published port so that web requests made on our local machine will reach the Rails server, running inside a container. Redis, however, doesn't need to be accessed externally; in fact, for security, we'd prefer it wasn't. By not exposing a port, it's hidden and inaccessible to the outside world.

We also don't specify any volumes to be mounted. The web service used volumes to mount our local directory containing our Rails project code inside the container. We did this so that, as we edit the files locally, the changes are automatically picked up inside the container too. For Redis, we don't need this behavior—we're not modifying any files.